

Laporan Praktikum Pekan 9

“Bagging & Random Forest (Feature Importance & Stability)”

Disusun Untuk Memenuhi Tugas Mata Kuliah MACHINE LEARNING

DOSEN PENGAMPU:

Derisma, M.T.



DISUSUN OLEH:

Karimah Irsyadiyah (2411533018)

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
T.A 2025/2026**

Daftar Isi

BAB I PENDAHULUAN.....	4
1.1 Latar Belakang Dasar Teori	4
1.2 Tujuan Praktikum.....	4
LINK GITHUB:.....	4
BAB II LANGKAH-LANGKAH PRAKTIKUM	5
File Praktikum8_PCA_Integration.ipynb	5
2.1 Import Library.....	5
2.2 Load Data Set.....	5
Bagian A – Decision Tree vs Random Fores	5
2.3 Decision Tree	5
2.4 Random Forest	6
Tabel Perbandingan:.....	6
Bagian B – Feature Importance	7
2.5 Feature Importance	7
2.6 Visualisasi Feature Importance	8
2.7 Analisis Feature Importance	8
1. Fitur Mana yang Paling Dominan?	8
2. Apakah Fitur Tersebut Intuitif Secara Medis?	9
3. Apakah Ada Fitur yang Hampir Tidak Berpengaruh?	9
Bagian C – Stability Test	9
2.8 Uji n_estimators	9
Mengapa PCA Tidak Banyak Membantu?.....	10
Apakah PCA Menghilangkan Interpretabilitas?	10
Bagian D – PCA + Random Forest	10
2.9 Terapkan PCA	10
2.10 Analisis PCA pada Random Fores:.....	11
1. Apakah PCA Meningkatkan Performa?	11
2. Mengapa Tree-Based Model Jarang Membutuhkan PCA?.....	11
3. Apakah PCA Menghilangkan Interpretabilitas?	11
TUGAS INDIVIDU PRAKTIKUM	12
1. Uji max_depth Berbeda pada Random Forest	12
2. Uji max_features	13
3. Uji min_samples_leaf	14
4. Perbandingan dengan Logistic Regression	14

5. Tabel Eksperimen.....	16
PERTANYAAN REFLEKTIF	16
1. Mengapa Random Forest lebih stabil dibanding single tree?	16
2. Apa perbedaan bias dan variance pada Tree vs RF?	16
3. Mengapa ensemble mengurangi overfitting?	17
4. Apakah Random Forest selalu lebih baik?.....	17
5. Kapan kita tetap memilih Decision Tree tunggal?	17

BAB I

PENDAHULUAN

1.1 Latar Belakang Dasar Teori

Ensemble Learning merupakan teknik machine learning yang menggabungkan beberapa model untuk menghasilkan performa yang lebih baik dibandingkan model tunggal. Salah satu metode ensemble yang populer adalah Bagging (*Bootstrap Aggregating*) yang digunakan pada algoritma Random Forest. Dengan menggabungkan banyak Decision Tree yang dilatih pada sampel data yang berbeda, Random Forest mampu meningkatkan stabilitas model dan mengurangi risiko overfitting.

Random Forest juga menyediakan informasi mengenai tingkat kepentingan setiap fitur (*feature importance*) sehingga dapat membantu dalam memahami faktor-faktor yang paling berpengaruh terhadap hasil prediksi. Pada praktikum ini dilakukan perbandingan antara Decision Tree dan Random Forest menggunakan Breast Cancer Dataset untuk menganalisis performa, stabilitas model, serta pengaruh PCA terhadap model berbasis pohon.

1.2 Tujuan Praktikum

1. Memahami konsep ensemble learning dan bagging.
2. Membandingkan performa Decision Tree dan Random Forest.
3. Menganalisis stabilitas model menggunakan variasi parameter.
4. Menginterpretasikan feature importance pada Random Forest.
5. Mengevaluasi pengaruh PCA terhadap model berbasis pohon.
6. Membandingkan performa model menggunakan cross validation.

LINK GITHUB:

https://github.com/iMaIrsdyh/KarimahIrsyadiyah_2411533018_ML2526/tree/main/PRAKTIKUM%209

BAB II

LANGKAH-LANGKAH PRAKTIKUM

File Praktikum8_PCA_Integration.ipynb

2.1 Import Library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

Penjelasan

Library yang digunakan meliputi NumPy dan Pandas untuk pengolahan data, Matplotlib untuk visualisasi, serta berbagai library Scikit-Learn untuk membangun model Decision Tree, Random Forest, PCA, dan evaluasi model menggunakan cross validation.

2.2 Load Data Set

```
data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Penjelasan

Dataset Breast Cancer dimuat dari Scikit-Learn dan dipisahkan menjadi fitur (X) serta target (y). Data kemudian dibagi menjadi data training dan testing dengan rasio 80:20 menggunakan `train_test_split()`.

Bagian A – Decision Tree vs Random Fores

2.3 Decision Tree

```
tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)
```

```
acc_tree_train = accuracy_score(y_train, tree.predict(X_train))
acc_tree_test = accuracy_score(y_test, tree.predict(X_test))

print("Decision Tree Train:", acc_tree_train)
print("Decision Tree Test :", acc_tree_test)
```

✓ 0.1s

```
Decision Tree Train: 1.0
Decision Tree Test : 0.9473684210526315
```

Penjelasan

Model Decision Tree dilatih menggunakan data training kemudian dievaluasi pada data training dan testing untuk melihat kemampuan generalisasi model.

Analisis

Nilai akurasi training mencapai 100%, sedangkan akurasi testing sebesar 94,74%. Hal ini menunjukkan bahwa Decision Tree cenderung menghafal data training sehingga berpotensi mengalami overfitting.

2.4 Random Forest

```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

acc_rf_train = accuracy_score(y_train, rf.predict(X_train))
acc_rf_test = accuracy_score(y_test, rf.predict(X_test))

print("Random Forest Train:", acc_rf_train)
print("Random Forest Test :", acc_rf_test)
```

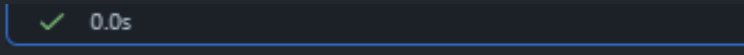
Analisis

Random Forest memperoleh akurasi testing sebesar 96,49%, lebih tinggi dibandingkan Decision Tree. Walaupun akurasi training tetap 100%, performa testing yang lebih tinggi menunjukkan bahwa Random Forest memiliki kemampuan generalisasi yang lebih baik.

Tabel Perbandingan:

```
## Tabel Perbandingan
comparison_table = pd.DataFrame({
    "Model": ["Decision Tree", "Random Forest"],
    "Train Accuracy": [acc_tree_train, acc_rf_train],
    "Test Accuracy": [acc_tree_test, acc_rf_test]
})
```

comparison_table



	Model	Train Accuracy	Test Accuracy
0	Decision Tree	1.0	0.947368
1	Random Forest	1.0	0.964912

Analisis Awal

- Decision Tree menunjukkan indikasi overfitting karena akurasi training sempurna namun testing lebih rendah.
- Random Forest lebih stabil dibandingkan Decision Tree.
- Gap train-test pada Random Forest lebih kecil sehingga kemampuan generalisasi lebih baik.

Bagian B – Feature Importance

2.5 Feature Importance

```
## Mengambil Feature Importance
importances = rf.feature_importances_

feat_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)

print(feat_importance.head(10))
```

	Feature	Importance
23	worst area	0.153892
27	worst concave points	0.144663
7	mean concave points	0.106210
20	worst radius	0.077987
6	mean concavity	0.068001
22	worst perimeter	0.067115
2	mean perimeter	0.053270
0	mean radius	0.048703
3	mean area	0.047555
26	worst concavity	0.031802

Penjelasan

Feature Importance menunjukkan kontribusi masing-masing fitur dalam proses pengambilan keputusan pada Random Forest.

2.6 Visualisasi Feature Importance

```
## Visualisasi
plt.figure(figsize=(8,6))
plt.barh(feat_importance["Feature"][:10],
         feat_importance["Importance"][:10])
plt.gca().invert_yaxis()
plt.title("Top 10 Feature Importance")
plt.show()
```

Penjelasan:

Kode di atas digunakan untuk memvisualisasikan sepuluh fitur dengan nilai *feature importance* tertinggi yang diperoleh dari model Random Forest. Grafik yang digunakan adalah grafik batang horizontal (*horizontal bar chart*) sehingga perbandingan kontribusi antar fitur dapat dilihat dengan lebih jelas.

Fungsi `plt.barh()` digunakan untuk membuat grafik batang horizontal, sedangkan `invert_yaxis()` digunakan agar fitur dengan nilai importance tertinggi ditampilkan pada bagian atas grafik. Judul grafik diberikan menggunakan `plt.title()` agar tujuan visualisasi lebih mudah dipahami.

Analisis:

Berdasarkan grafik *Top 10 Feature Importance*, terlihat bahwa beberapa fitur memiliki kontribusi yang jauh lebih besar dibandingkan fitur lainnya. Fitur dengan nilai importance tertinggi merupakan fitur yang paling sering digunakan oleh Random Forest dalam proses pembentukan pohon keputusan.

Grafik juga menunjukkan bahwa tidak semua fitur memiliki pengaruh yang sama terhadap proses klasifikasi. Beberapa fitur memiliki nilai importance yang sangat tinggi, sedangkan fitur lainnya memiliki kontribusi yang relatif kecil. Hal ini menunjukkan bahwa model Random Forest secara otomatis mampu mengidentifikasi fitur-fitur yang paling relevan dalam membedakan tumor ganas dan tumor jinak pada Breast Cancer Dataset.

2.7 Analisis Feature Importance

1. Fitur Mana yang Paling Dominan?

Berdasarkan hasil Random Forest, fitur dengan nilai **feature importance** tertinggi adalah **worst area**, diikuti oleh **worst concave points**, **mean concave points**, dan **worst radius**. Fitur-fitur tersebut memiliki kontribusi terbesar dalam proses pengambilan keputusan model saat membedakan tumor jinak dan ganas.

2. Apakah Fitur Tersebut Intuitif Secara Medis?

Ya. Secara medis, ukuran area, radius, dan tingkat cekungan (*concavity*) sel merupakan indikator penting dalam diagnosis kanker payudara. Tumor ganas umumnya memiliki ukuran yang lebih besar serta bentuk yang lebih tidak beraturan dibandingkan tumor jinak. Oleh karena itu, hasil feature importance yang diperoleh sesuai dengan karakteristik medis dari dataset Breast Cancer.

3. Apakah Ada Fitur yang Hampir Tidak Berpengaruh?

Ya. Beberapa fitur memiliki nilai importance yang sangat kecil dibandingkan fitur utama. Fitur-fitur tersebut jarang digunakan dalam proses pemisahan node pada Random Forest sehingga kontribusinya terhadap prediksi relatif rendah. Walaupun demikian, fitur tersebut tetap dapat memberikan informasi tambahan ketika dikombinasikan dengan fitur lainnya.

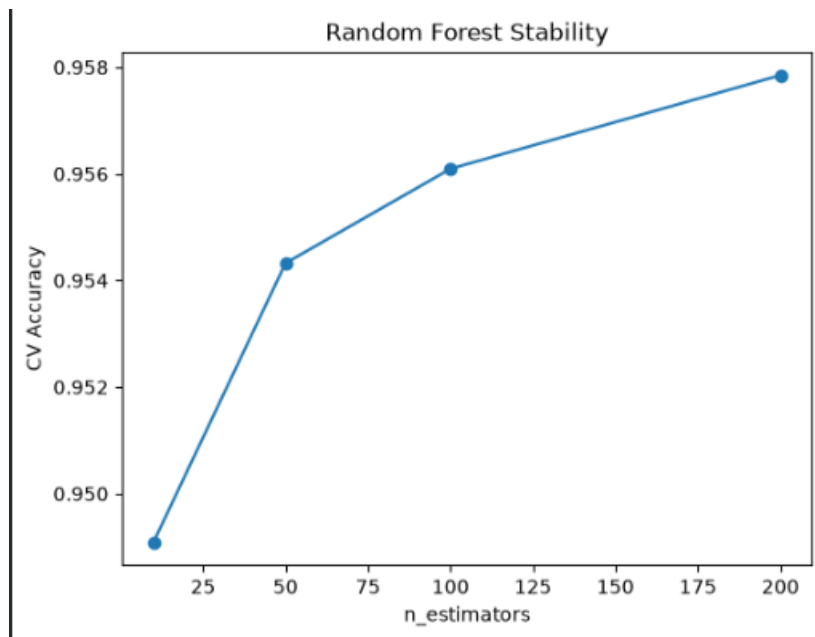
Bagian C – Stability Test

2.8 Uji n_estimators

```
## Bagian C - Stability Test
## Uji n_estimators
estimators = [10, 50, 100, 200]
rf_scores = []

for n in estimators:
    model = RandomForestClassifier(n_estimators=n, random_state=42)
    score = cross_val_score(model, X, y, cv=5).mean()
    rf_scores.append(score)

plt.plot(estimators, rf_scores, marker='o')
plt.xlabel("n_estimators")
plt.ylabel("CV Accuracy")
plt.title("Random Forest Stability")
plt.show()
```



Analisis:

Hasil menunjukkan bahwa penggunaan PCA justru sedikit menurunkan performa Random Forest.

Mengapa PCA Tidak Banyak Membantu?

Random Forest tidak menggunakan perhitungan jarak seperti KNN atau SVM sehingga tidak terlalu terpengaruh oleh jumlah dimensi fitur.

Apakah PCA Menghilangkan Interpretabilitas?

Ya. Setelah PCA diterapkan, fitur asli berubah menjadi komponen utama sehingga interpretasi feature importance menjadi lebih sulit dilakukan.

Bagian D – PCA + Random Forest

2.9 Terapkan PCA

```
## Bagian D – PCA + Random Forest
## PCA dan Random Forest
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=10)
X_pca = pca.fit_transform(X_scaled)

rf_pca = RandomForestClassifier(n_estimators=100, random_state=42)
score_rf_pca = cross_val_score(rf_pca, X_pca, y, cv=5).mean()

score_rf = cross_val_score(rf, X, y, cv=5).mean()
```

```
print("RF tanpa PCA:", score_rf)
print("RF dengan PCA:", score_rf_pca)
```

✓ 1.9s

RF tanpa PCA: 0.9560937742586555

RF dengan PCA: 0.9507529886663562

2.10 Analisis PCA pada Random Fores:

1. Apakah PCA Meningkatkan Performa?

Berdasarkan hasil praktikum:

Terlihat bahwa penggunaan PCA justru sedikit menurunkan performa Random Forest. Hal ini menunjukkan bahwa reduksi dimensi tidak memberikan keuntungan pada model ini untuk dataset Breast Cancer.

2. Mengapa Tree-Based Model Jarang Membutuhkan PCA?

Model berbasis pohon seperti Decision Tree dan Random Forest tidak menggunakan perhitungan jarak antar data seperti KNN atau SVM. Model ini secara otomatis melakukan pemilihan fitur saat proses pembentukan pohon sehingga mampu menangani data berdimensi tinggi dengan baik tanpa perlu reduksi dimensi menggunakan PCA.

3. Apakah PCA Menghilangkan Interpretabilitas?

Ya. Salah satu keunggulan Random Forest adalah kemampuan untuk menghitung feature importance berdasarkan fitur asli dataset. Setelah PCA diterapkan, fitur-fitur asli diubah menjadi komponen utama (*principal components*) yang merupakan kombinasi dari banyak fitur. Akibatnya interpretasi hasil model menjadi lebih sulit karena komponen PCA tidak memiliki makna fisik yang langsung dapat dijelaskan.

TUGAS INDIVIDU PRAKTIKUM

1. Uji max_depth Berbeda pada Random Forest

```
## TUGAS INDIVIDU ##
# Uji max_depth Berbeda pada Random Forest
depths = [3, 5, 10, 15, None]

depth_scores = []

for d in depths:

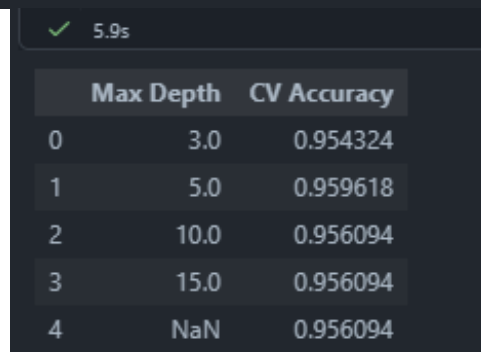
    model = RandomForestClassifier(
        n_estimators=100,
        max_depth=d,
        random_state=42
    )

    score = cross_val_score(
        model,
        X,
        y,
        cv=5
    ).mean()

    depth_scores.append(score)

depth_table = pd.DataFrame({
    "Max Depth": depths,
    "CV Accuracy": depth_scores
})

depth_table
```



	Max Depth	CV Accuracy
0	3.0	0.954324
1	5.0	0.959618
2	10.0	0.956094
3	15.0	0.956094
4	NaN	0.956094

Analisis

Performa terbaik diperoleh pada **max_depth = 5** dengan akurasi 95,96%. Hal ini menunjukkan bahwa pembatasan kedalaman pohon dapat membantu meningkatkan kemampuan generalisasi model.

2. Uji max_features

```
## Uji Parameter max_features
max_features_values = [
    "sqrt",
    "log2",
    None
]

feature_scores = []

for mf in max_features_values:

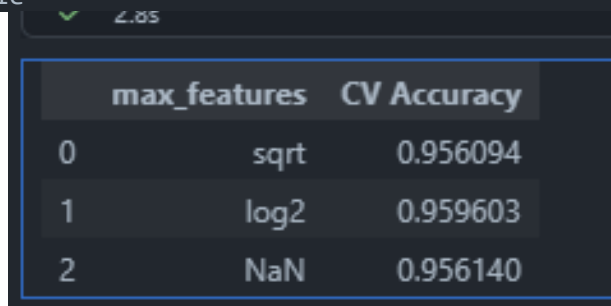
    model = RandomForestClassifier(
        n_estimators=100,
        max_features=mf,
        random_state=42
    )

    score = cross_val_score(
        model,
        X,
        y,
        cv=5
    ).mean()

    feature_scores.append(score)

max_feature_table = pd.DataFrame({
    "max_features": max_features_values,
    "CV Accuracy": feature_scores
})

max_feature_table
```



	max_features	CV Accuracy
0	sqrt	0.956094
1	log2	0.959603
2	NaN	0.956140

Analisis:

Nilai terbaik diperoleh pada **max_features = log2** dengan akurasi 95,96%.

3. Uji min_samples_leaf

```
## Uji Parameter min_samples_leaf
leaf_values = [1, 2, 4, 8]

leaf_scores = []

for leaf in leaf_values:

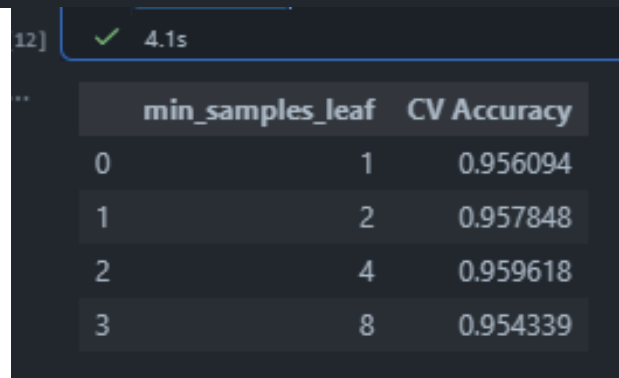
    model = RandomForestClassifier(
        n_estimators=100,
        min_samples_leaf=leaf,
        random_state=42
    )

    score = cross_val_score(
        model,
        X,
        y,
        cv=5
    ).mean()

    leaf_scores.append(score)

leaf_table = pd.DataFrame({
    "min_samples_leaf": leaf_values,
    "CV Accuracy": leaf_scores
})

leaf_table
```



	min_samples_leaf	CV Accuracy
0	1	0.956094
1	2	0.957848
2	4	0.959618
3	8	0.954339

Analisis:

Performa terbaik diperoleh pada **min_samples_leaf = 4** dengan akurasi 95,96%.

4. Perbandingan dengan Logistic Regression

```
## Perbandingan dengan Logistic Regression
from sklearn.linear_model import LogisticRegression
```

```

log_model = LogisticRegression(
    max_iter=5000
)

score_log = cross_val_score(
    log_model,
    X,
    y,
    cv=5
).mean()

score_rf = cross_val_score(
    rf,
    X,
    y,
    cv=5
).mean()

comparison_model = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Random Forest"
    ],
    "CV Accuracy": [
        score_log,
        score_rf
    ]
})

comparison_model

```

	Model	CV Accuracy
0	Logistic Regression	0.950800
1	Random Forest	0.956094

Analisis:

Random Forest memperoleh performa yang lebih baik dibandingkan Logistic Regression karena mampu menangkap hubungan non-linear antar fitur.

5. Tabel Eksperimen

```
## Tabel Eksperimen
experiment_table = pd.DataFrame({
    "Eksperimen": [
        "Random Forest Default",
        "RF max_depth terbaik",
        "RF max_features terbaik",
        "RF min_samples_leaf terbaik",
        "Logistic Regression"
    ],
    "CV Accuracy": [
        score_rf,
        max(depth_scores),
        max(feature_scores),
        max(leaf_scores),
        score_log
    ]
})

experiment_table
```

Analisis:

Konfigurasi terbaik diperoleh dari pengaturan **max_depth = 5** dan **min_samples_leaf = 4** dengan akurasi sebesar 95,96%.

PERTANYAAN REFLEKTIF

1. Mengapa Random Forest lebih stabil dibanding single tree?

Random Forest lebih stabil karena terdiri dari banyak Decision Tree yang dilatih menggunakan sampel data dan subset fitur yang berbeda. Prediksi akhir diperoleh melalui mekanisme voting dari seluruh pohon sehingga kesalahan yang terjadi pada satu pohon dapat dikompensasi oleh pohon lainnya. Akibatnya, Random Forest menghasilkan prediksi yang lebih konsisten dan memiliki kemampuan generalisasi yang lebih baik dibandingkan Decision Tree tunggal.

2. Apa perbedaan bias dan variance pada Tree vs RF?

Decision Tree umumnya memiliki bias yang rendah karena mampu mempelajari pola data dengan sangat detail, tetapi memiliki variance yang tinggi karena sangat sensitif terhadap perubahan data training. Sebaliknya, Random Forest tetap mempertahankan bias yang relatif rendah namun mampu menurunkan variance melalui penggabungan banyak

pohon. Oleh karena itu, Random Forest biasanya menghasilkan model yang lebih stabil dan tidak mudah overfit.

3. Mengapa ensemble mengurangi overfitting?

Ensemble mengurangi overfitting karena prediksi tidak hanya bergantung pada satu model. Pada Random Forest, setiap pohon memiliki karakteristik dan kesalahan yang berbeda. Ketika hasil seluruh pohon digabungkan melalui voting, kesalahan individu akan saling menutupi sehingga model menjadi lebih robust terhadap noise dan data tertentu yang mungkin menyebabkan overfitting pada model tunggal.

4. Apakah Random Forest selalu lebih baik?

Tidak selalu. Meskipun Random Forest sering memberikan akurasi yang lebih tinggi, model ini membutuhkan waktu komputasi dan memori yang lebih besar dibandingkan Decision Tree tunggal. Selain itu, interpretasi model juga menjadi lebih sulit karena terdiri dari banyak pohon. Pada dataset yang sederhana atau ketika interpretabilitas menjadi prioritas utama, Decision Tree dapat menjadi pilihan yang lebih sesuai.

5. Kapan kita tetap memilih Decision Tree tunggal?

Decision Tree tunggal lebih cocok digunakan ketika interpretasi model sangat penting, misalnya pada bidang kesehatan, bisnis, atau pengambilan keputusan yang membutuhkan penjelasan yang mudah dipahami. Selain itu, Decision Tree lebih cepat dilatih, membutuhkan sumber daya komputasi yang lebih kecil, dan mudah divisualisasikan sehingga cocok digunakan untuk analisis awal atau dataset yang tidak terlalu kompleks.